pfi

Python for Fixed Income

COURSE 2 · BOND MATH

Case Study: A New Issue Calendar

WEEK 1 · DAY 5

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Eight new issues, one morning

Settling 30 September 2026, semiannual, 30/360, redemption 100

Issuer	Rating	Coupon %	Years	Re-offer
Thalostrin Finance	BBB+	5.125	5	99.874
Galvostrin Resources	BBB+	5.625	10	99.512
Hulvimer Pipeline	BBB	5.500	7	99.760
Kezerone Paper	A-	4.625	3	99.942
Elvorin Medical	BB-	7.875	8	100.000
Corvolvey Systems	BB+	7.000	5	99.000
Rulvolvey Industrial	BBB-	6.000	30	98.975
Morvithe Logistics	B	8.750	6	98.500

Invented issuers and numbers. The day describes the calendar and never says an issue is cheap or rich.

Week 1: price and yield

Day 1	Time value of money
Day 2	Cash flows and price from yield
Day 3	The price-yield relationship
Day 4	Yield from price
Day 5	**Case study: a new issue calendar**

Re-offer price: what investors pay for the new bonds, per 100 of face.

The job: describe the calendar

Context

Each new bond settles on its issue date: a coupon date, nothing accrued. The functions of days 2 and 4 apply exactly.

Objective, with your own module

1. Yields from the re-offer prices
2. Every issue at one common yield, 5.50 percent
3. The coupon on the 1/8 grid that prices nearest par
4. What one eighth of coupon is worth
5. All of it as DataFrame columns, checked against Excel
6. A workbook of live YIELD and PRICE formulas

Six Predict questions

Each shows a question and three answers before the code runs.

1. Read the question and the options

2. Decide: A, B, or C

3. Run the cell, then read why

The answers come from running the code, not from a view on any issuer.

Yields from re-offer prices

```
# one issue at a time, with zip
calendar["yield_pct"] = [
    bondmath.yield_from_price(c, p, t)
    for c, p, t in zip(coupons, prices, years)]
```

```
    Thalostrin: 5.125 coupon at 99.874  ->  5.1539 percent
```

Below 100, the yield is above the coupon: day 3's rule, read from the price.

Predict 1

Corvolvey, 7% for 5 years at 99.000, and Rulvolvey, 6% for 30 years at 98.975. Whose yield is further above its coupon?

A) Rulvolvey: its discount is slightly larger

B) About the same: the discounts are about the same

C) Corvolvey: its discount is spread over fewer years

Excel side by side (tested in Excel)

`=YIELD(DATE(2026,9,30),DATE(2031,9,30),5.125/100,99.874,100,2,0)`	0.0515390782041645

Every issue at one yield

Re-offer yields differ by credit and by term at once. Price every issue at one yield, 5.50 percent, to see what the coupon and the term do on their own.

Coupon against 5.50	Price at 5.50
above	above 100
equal (Hulvimer)	100
below	below 100

5.50 is a round number for the exercise, not a market level for any issuer. The prices are what-ifs.

Predict 2

Corvolvey pays 7% for 5 years. Rulvolvey pays 6% for 30 years. Priced at the same 5.50 percent, which price is higher?

A) Corvolvey: its coupon is a point higher

B) Rulvolvey, even with the lower coupon

C) The same: they share one yield

A small extra for a long time

Issue	Coupon %	Extra a half-year	Half-years	Price at 5.50
Corvolvey	7.00	0.750	10	106.4801
Rulvolvey	6.00	0.250	60	107.3057

The price above 100 is today's value of all the extra coupons. A small extra for 60 half-years beats a large one for 10.



Excel side by side (tested in Excel)

```
=PRICE(DATE(2026,9,30),DATE(2056,9,30),6/100,5.5/100,100,2,0)
```

107.30566550271652

The coupon nearest par on a 1/8 grid

Coupons are set in eighths of a percent. A coupon equal to the yield prices at 100, but a yield is rarely a whole number of eighths.

5.500	5.625	5.750	5.875

Galvostrin yields 5.6897: between 5.625 and 5.750

The direct way: price the two grid coupons either side of the yield, at that yield, and keep the price nearer 100.

Predict 3

Galvostrin, 10 years, yields 5.6897. 5.625 is 6.5 bp below, 5.750 is 6.0 bp above. Which prices nearer 100?

A) 5.625, the coupon on the calendar

B) 5.750, the grid coupon nearer the yield

C) Neither: the two are equally far from 100

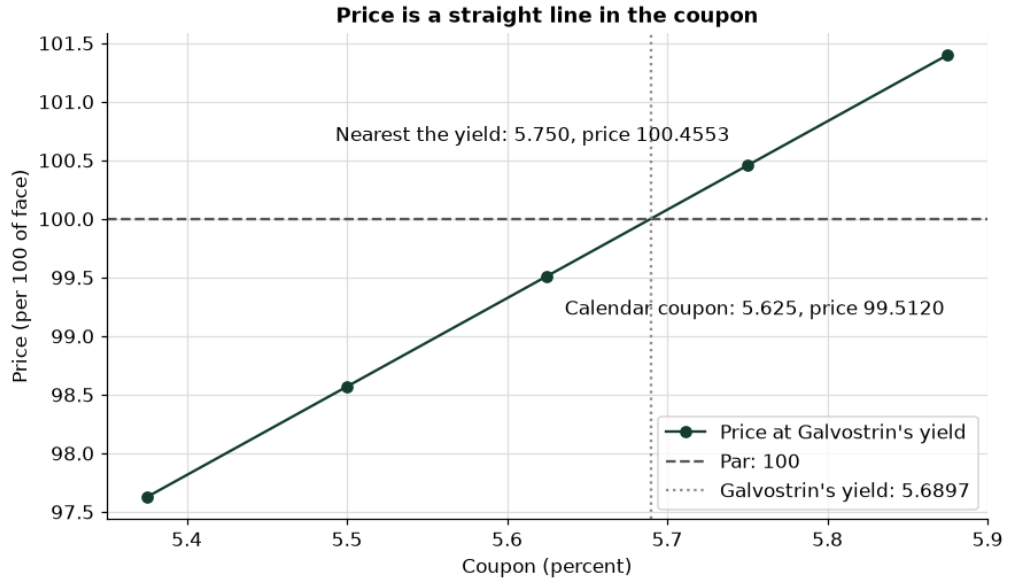
Price is a straight line in the coupon

Galvostrin coupon	Price at its yield	From 100
5.625	99.5120	0.4880
5.750	100.4553	0.4553

Each eighth of coupon adds the same amount of price. The line crosses 100 at the yield, so the grid coupon nearest the yield prices nearest par.

Excel side by side (tested in Excel)

`=MROUND(5.69,0.125)`	5.75



Four issues end up one to three eighths above the calendar's coupon; the other four keep theirs.

What is one eighth of coupon worth?

An eighth of a percent a year is 0.0625 per 100 of face every half-year, for the life of the bond.

```
# the step, at the common yield
price_from_yield(c + 0.125, 5.5, years)
  - price_from_yield(c, 5.5, years)
```

Each extra 0.0625 is paid later, so each is discounted, as on day 1.

Predict 4

Rulvolvey is a 30-year bond. At 5.50 percent, how much does its price rise with a coupon one eighth higher?

A) 0.125, one eighth of a point

B) About 1.8: less than an eighth a year

C) 3.75: an eighth for each of 30 years

The calendar as a DataFrame

Every result went into the calendar as a column, one issue at a time. Why not hand the whole columns to the function?

```
bondmath.yield_from_price(calendar["coupon_pct"],
    calendar["reoffer_price"], calendar["years"])
```

Two tests for two facts the calendar relied on

`test_par_bond_prices_at_100`

coupon equal to yield: 100, every term and frequency

`test_price_falls_as_yield_rises`

six yields from 2 to 12 percent

Predict 5

The cell calls yield_from_price once, with three whole columns instead of three numbers. What happens?

A) A column of eight yields

B) An error

C) One yield, for the first issue

13 passed

days 1 to 4, and today's two tests

A workbook of live formulas

Column	Holds	Row 2
A to E	the inputs	`cusip, issuer, coupon, maturity, re-offer`
F	Python's yield	`a value`
G	Excel's yield	`=YIELD(DATE(2026,9,30),D2,C2/100,E2,100,2,0)*100`
H	the gap	`=G2-F2`
I	Python's price	`a value`
J	Excel's price	`=PRICE(DATE(2026,9,30),D2,C2/100,5.5/100,100,2,0)`
K	the gap	`=J2-I2`

Written with openpyxl: a cell whose text starts with = is written as a formula.

Saved in the work folder, never in the course repo.

Predict 6

Read the saved workbook back with pd.read_excel. What does the excel_yield_pct column hold?

A) The yields, as Excel shows them

B) The formula text

C) Nothing: NaN

pandas reads no numbers; Excel calculates

pd.read_excel: NaN

openpyxl writes a formula's text and no result. pandas reads the result Excel would store beside it, and Excel has never seen the file.

Only a spreadsheet program calculates a formula.

Opened in Excel (tested)

The YIELD and PRICE columns showed their values as soon as the file opened. Change a re-offer price, and its yield and gap change with it.

-6.1E-09

largest yield gap, percent

1.3E-13

largest price gap

openpyxl writes fullCalcOnLoad into the file: calculate everything on opening. Tolerance: 1e-6.

Today's Excel, every formula tested

Job	Excel (tested)	Returns
Yield from re-offer	`=YIELD(DATE(2026,9,30),DATE(2031,9,30),5.125/100,99.874,100,2,0)`	0.051539
Price at 5.50	`=PRICE(DATE(2026,9,30),DATE(2056,9,30),6/100,5.5/100,100,2,0)`	107.305666
One eighth	`=PRICE(...,6.125/100,5.5/100,...)-PRICE(...,6/100,5.5/100,...)`	1.826416
Nearest 1/8	`=MROUND(5.69,0.125)`	5.75
Workbook yield	`=YIELD(DATE(2026,9,30),D2,C2/100,E2,100,2,0)*100`	percent

YIELD and PRICE take rates as decimals, written /100. YIELD returns a decimal, so the workbook multiplies it by 100.

All 34 calendar rows of the reference file agree with bondmath: 13 YIELD rows and 21 PRICE rows.

End of week 1: git tag

Step	Command	What you see
1	git status --short	M test_bondmath.py, and ?? new_issue_calendar.xlsx
2	git add, git commit	today's two tests, committed by name
3	git tag week1	nothing printed
4	git tag	week1
5	git log --oneline --decorate	(HEAD -> main, tag: week1)

A tag is a name on one commit, and it never moves.

The workbook stays out of the commit: the notebook makes it from the module, so it is not saved in Git.

Bring it home: paste, test, commit, tag

1

Your folder

Set-Location to my_bondmath, in your projects folder.

2

Paste

Today's test cell at the end of test_bondmath.py, without the %%writefile line.

3

Test

python -m pytest -q: 13 passed, or 17 with the exercises of days 1 to 4.

4

Commit and tag

git add, git commit, git tag week1, git log --oneline --decorate.

5

Open Excel

Run the Invoke-Item line the notebook printed, and read the gaps.

bondmath.py does not change today. Copy fallen behind? Take the start-of-day files from today's setup cell, and git diff shows what changed.

In Colab: download test_bondmath.py and the workbook. In Protected View, choose Enable Editing before you compare.

The AI check

The docstring is the prompt. The code that comes back runs, and has one convention error.

1

Docstring

Units, convention, and the Excel formula: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert against Excel's reference rows. Run it: it fails.

→

4

Fix

Find the convention it broke, fix it, run the test again.

Today: price_from_yield works per 1 of face, so every price comes back a hundredth of Excel's.

Code that runs is not the same as code that is right.

What you can do now

- Turn re-offer prices into yields, and price a calendar at one yield
- Find the grid coupon nearest par, and what an eighth of coupon is worth
- Apply the module one bond at a time, and say why
- Write a workbook of live YIELD and PRICE formulas with openpyxl
- Name the end of a week with git tag

Week 2, Day 1 Dates and coupon schedules: a bond that settles between two coupons